Hermes Agent Server: An Operations-First Primer

Mick Cooney

27 September 2026

Table of Contents

1 What you are operating **2**
1.1 What This Primer Covers . . . 3
1.2 What This Primer Is Not . . . 3
1.3 What Hermes Agent is used for . . . 3
1.4 Common tasks . . . 3
1.5 The boundaries that matter . . . 4
1.6 Choose a surface and starting setup . . . 4
1.6.1 A common failure sequence . . . 5
1.7 Choose a starting setup . . . 5
1.7.1 Start small . . . 5
1.7.2 Add a client or channel deliberately . . . 5
1.7.3 Select the right expansion . . . 5
1.8 Hermes architecture in one mental model . . . 6
1.8.1 The process relationships . . . 7

2 Install and prove one local run **7**
2.1 Source checkout for development . . . 8

3 Configure providers and durable state **8**
3.1 Providers and models . . . 9
3.2 Provider fallback and model overrides . . . 10
3.3 Memory, skills, and durable state . . . 10
3.3.1 Memory: curated continuity, not an infinite transcript . . . 10
3.3.2 Skills: procedural knowledge with a write boundary . . . 11

4 Set the execution boundary **12**
4.1 Security baseline . . . 13
4.2 Profiles: separate agents without separate installs . . . 13

5 Add one ingress surface **14**
5.1 Messaging gateway: `hermes gateway` . . . 14
5.2 OpenAI-compatible API . . . 15
5.3 Desktop and `hermes serve` . . . 16

6 Deploy and supervise **17**
6.1 Run Hermes in a container . . . 17
6.1.1 Podman . . . 18
6.1.2 Compose shape . . . 18

7 Operate and diagnose **19**

8 Backup, restore, and recover **20**
8.1 Full machine backup . . . 20
8.1.1 Profile handoff . . . 21
8.1.2 Recovery checklist . . . 21

9 Reference and alternatives **21**
9.1 When Hermes is the better fit . . . 22
9.2 When OpenClaw is the better fit . . . 22
9.3 When they can be combined . . . 23
9.4 Operational reference . . . 23
9.4.1 Surface-selection checklist . . . 23
9.4.2 Hardening checklist . . . 23
9.4.3 Troubleshooting map . . . 23
9.5 Reference links . . . 24
9.6 Closing perspective . . . 25

1 What you are operating

Hermes Agent is easy to misclassify. It is described as a personal AI assistant, a coding agent, a messaging bot, a desktop application, an OpenAI-compatible server, and a self-improving agent runtime. Those descriptions point at real parts of the project, but they do not describe the same process. The most expensive operational mistakes come from treating the commands as interchangeable.

This primer resolves that ambiguity first. `hermes gateway` is the long-running messaging gateway. The OpenAI-compatible API server is an HTTP interface that normally listens on `127.0.0.1:8642` when enabled and is normally started alongside the gateway. `hermes serve` is the headless JSON-RPC/WebSocket backend used by Hermes Desktop and remote desktop-style clients; it is commonly reached on port `9119`. These are related Hermes surfaces, but starting one does not automatically start the others.

The official project is NousResearch/hermes-agent, documented at hermes-agent.nousresearch.com. This document describes the upstream project as checked on 27 September 2026. Hermes changes quickly, so treat command help and the linked upstream pages as the authority when a local installation disagrees with an example here.

1.1 What This Primer Covers

You will learn how its processes fit together, how to install and configure one safe profile, where it keeps state, how providers, models, memory, skills, profiles, and terminal backends work, and how to operate it safely. You will also get runnable examples for the three server surfaces, container deployment patterns, backup and recovery runbooks, and a comparison with OpenClaw.

The emphasis is on the operational boundary: what receives input, what chooses a model, what can execute commands, what persists, and what must be protected.

1.2 What This Primer Is Not

This is not a model benchmark, a complete catalogue of every messaging adapter, or a replacement for provider documentation. Hermes Agent is not the same thing as the Nous Hermes model family; here, "Hermes" means the agent runtime and its server surfaces. The examples assume a technically literate operator who can read shell commands, YAML, and HTTP requests. They do not assume that a model can safely execute any action it proposes: tool policy, filesystem permissions, network exposure, and human approval remain your responsibility.

1.3 What Hermes Agent is used for

Hermes Agent is an open-source, tool-using agent runtime from Nous Research. It puts a model in a loop with tools, persistent instructions, session history, skills, and optional memory providers. You can use it interactively in a terminal, expose it through a desktop application, connect it to messaging platforms, or embed it behind an OpenAI-shaped HTTP client.

The important mental model is not "a chatbot with many frontends." It is "one agent runtime with several ingress and control surfaces." A Telegram message, a CLI prompt, an OpenAI-compatible request, and a Desktop chat can all reach an agent configured with the same profile, but each surface has a different protocol, lifecycle, authentication story, and set of assumptions about the caller.

1.4 Common tasks

Hermes can draft and revise documents, inspect repositories, run development commands, search the web, use browser or computer-use tools, remember compact user preferences, follow procedural skills, and send results back through a messaging channel. A profile can be shaped into a coding assistant, research assistant, homelab operator, customer-support bot,

or narrowly scoped internal worker.

The value increases when the task has continuity. A one-shot completion does not need session storage, a profile, or a gateway. A coding assistant that should remember repository conventions, use the same terminal workspace, and answer from Telegram does.

1.5 The boundaries that matter

Hermes is not a model server in the sense of Ollama, vLLM, or llama.cpp. It consumes providers and model endpoints; it does not replace the inference engine behind them. It can expose an OpenAI-compatible API, but that does not mean it serves model weights or becomes a general-purpose inference router.

Hermes is not only a desktop UI. Desktop launches or connects to a headless `hermes serve` backend. The Electron/React surface is a client of that backend, not the agent runtime itself.

Hermes is not the messaging gateway. The gateway is one long-running process within the larger system. Running `hermes serve` will not connect Telegram or Discord; running `hermes gateway` will not make the Desktop JSON-RPC backend appear on port 9119.

Hermes is not a security boundary by itself. Model output is untrusted input. The terminal backend, OS account, container permissions, API bind address, messaging allowlists, and review gates determine the blast radius.

1.6 Choose a surface and starting setup

Use this table when someone says "the Hermes server" to distinguish which process and protocol a client needs. Each surface exposes a different part of the runtime.

Surface	Process	Transport	Typical client	Distinct from
Messaging gateway	`hermes gateway`	Messaging adapters	Telegram, Discord, Slack, WhatsApp	Desktop backend
OpenAI-compatible API	Gateway with API enabled	HTTP, usually `:8642`	Open WebUI, SDKs, automation	Model inference; Desktop WebSocket
Headless backend	`hermes serve`	JSON-RPC/WebSocket, usually `:9119`	Hermes Desktop	Messaging adapters

The official API Server guide documents the HTTP surface. The Desktop guide describes the headless `hermes serve` process and its `tui_gateway` JSON-RPC/WebSocket API. The gateway internals guide describes the messaging process.

1.6.1 A common failure sequence

Suppose Desktop says it cannot connect, but Telegram works. Do not restart the Telegram gateway repeatedly: inspect the `hermes serve` process, its port, its authentication, and the Desktop connection target. Conversely, if Desktop works but Telegram is silent, inspect `hermes gateway status`, platform credentials, pairing, and adapter logs. If Open WebUI reports connection refused on `8642`, check that the API server is enabled and that you did not start only `hermes serve`.

The port numbers are defaults and conventions, not a substitute for observing the process startup line. Profiles and container deployments can require distinct API ports. A Desktop backend may choose an available port when launched by the app. Always use the printed readiness line or the configured bind address as the final answer.

1.7 Choose a starting setup

1.7.1 Start small

Start with the CLI or terminal UI on one machine, one profile, one provider, and the local terminal backend. This gives you a small failure surface while you learn how the agent uses tools, writes memory, and edits skills. It is the right place to establish a safe working directory and decide whether local execution is acceptable.

1.7.2 Add a client or channel deliberately

Hermes Desktop provides a richer chat surface while `hermes serve` owns the agent runtime. The backend can be local or remote. A remote setup commonly runs `hermes serve` on a server reachable over a private network or VPN, while the desktop client connects to its JSON-RPC/WebSocket endpoint. Messaging remains separate: start a gateway independently if you also want Telegram or Discord.

1.7.3 Select the right expansion

The API server lets an existing OpenAI-shaped client use Hermes as an agent backend. The gateway connects messaging adapters and scheduled jobs. A profile separates credentials, identity, skills, memory, sessions, cron jobs, and gateway state. Add each deliberately: a bot that can run shell commands is not safe merely because its chat channel is private.

1.8 Hermes architecture in one mental model

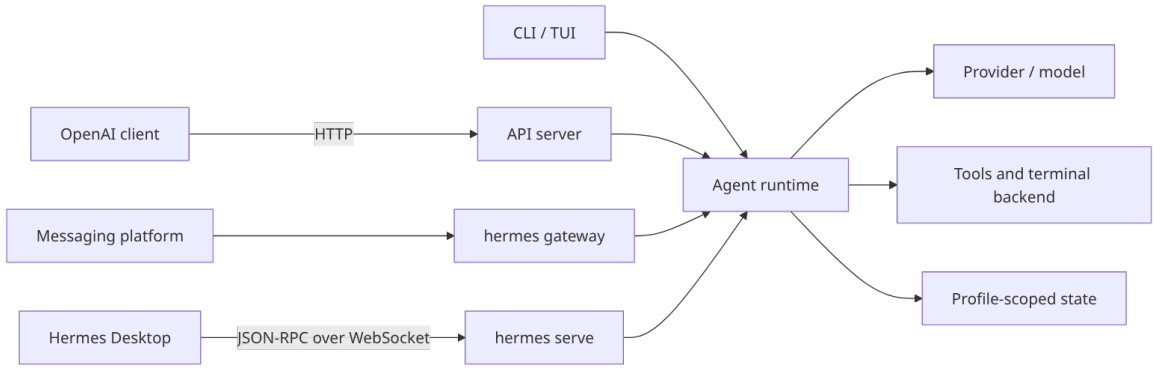
Think in layers: client surface, transport, agent loop, provider, tools, and state.

The client surface is the command-line/terminal user interface (CLI/TUI), Desktop, dashboard, messaging adapter, or API client. The transport layer turns that interaction into a local call, HTTP request, platform event, or JSON-RPC/WebSocket message. JSON-RPC is a request/response protocol; WebSocket is the persistent connection that carries it for Desktop clients. The agent loop assembles instructions and history, chooses a provider/model, asks for the next step, validates tool calls, executes approved tools, and repeats until it can answer. The provider layer handles model authentication and protocol differences. The tool layer includes terminal execution, web and browser capabilities, file operations, skills, memory, MCP integrations, and platform-specific actions. The state layer persists configuration, credentials, sessions, memory, skills, logs, and profile metadata.

This model prevents category errors. A malformed tool call is usually a provider/model or tool-schema issue, not a gateway-port issue. A message from an unknown Telegram user is an authorization issue, not a memory issue. A Desktop connection refusal is a backend lifecycle or network issue, not proof that the model provider is down.

1.8.1 The process relationships

The normal relationships between clients, Hermes surfaces, and the agent runtime are:



The gateway and `serve` share runtime concepts and state conventions, but they are separate long-running surfaces. A server may run both, but do not infer that one command supervises the other unless your service manager or container definition explicitly does so.

2 Install and prove one local run

The supported installation path is documented in Installation. On non-Windows systems the installer expects Git; Linux installs may also need `curl` and `xz-utils`. The installer provisions its managed Python and Node-related dependencies rather than asking you to build a hand-maintained system environment.

Run the official installer from a shell you trust. If you prefer to inspect downloaded scripts before execution, fetch the script to a temporary file, read it, and then run it according to the upstream instructions.

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

For a headless host that does not need browser automation, the official installer supports skipping the browser setup:

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash -s -- --skip-browser
```

Verify the launcher before configuring a provider:

```
hermes --version
hermes doctor
```

The interactive setup wizard covers model, terminal, gateway, tool, and agent configuration. The Portal path is the shortest way to configure a provider plus the Nous tool gateway when you have a Portal account:

```
hermes setup --portal
```

For a deliberately constrained first run, choose the Blank Slate setup when the wizard offers it. It starts with the minimum provider/model, file-operations, and terminal toolsets; web,

browser, code execution, memory, delegation, cron, skills, plugins, and MCP remain disabled until you opt in. Hermes requires a model context window of at least 64K tokens.

For a more targeted setup, use the surface-specific commands:

```
hermes model
hermes config set terminal.backend docker
hermes gateway setup
hermes tools
```

The TUI is the recommended interactive surface. Establish four facts before enabling a gateway or remote client: which profile is active, which provider/model answers a simple prompt, where terminal commands execute, and whether sessions resume.

```
hermes --tui
```

Use a harmless, observable prompt such as "Reply with the active profile and configured model; do not run a command." If tools are enabled, perform a read-only check and inspect the configured backend and working directory yourself. Then close the session and verify persistence:

```
hermes --continue
hermes sessions list
```

If this acceptance test fails, fix the provider, profile, or session configuration before adding messaging, API, Desktop, or automation surfaces.

2.1 Source checkout for development

If you are modifying Hermes itself, use the repository's development instructions and its managed environment. A source checkout is not the same as a production installation: the installed launcher must use the project virtual environment, and updates may alter both Python and Node dependencies. Run `hermes doctor` after changes and pin the commit or image tag for reproducible deployments.

3 Configure providers and durable state

The default Hermes home is `~/.hermes`. A profile changes the effective home through `HERMES_HOME`; the same layout then appears under `~/.hermes/profiles/<name>/`. The Configuration guide gives the current directory map.

```
~/.hermes/
|-- config.yaml       # non-secret behavior and model/tool settings
|-- .env              # API keys, bot tokens, and other secrets
|-- auth.json         # OAuth/provider credentials when used
|-- SOUL.md           # primary identity and operating instructions
|-- memories/         # MEMORY.md and USER.md
|-- skills/           # bundled, installed, and agent-created skills
|-- sessions/         # session data and transcripts
|-- state.db          # canonical SQLite session/state store where present
|-- cron/             # scheduled job definitions and state
```

```
|-- logs/             # agent, gateway, and error logs
|-- plugins/          # installed Hermes plugins
`-- profiles/         # named profile homes
```

The exact tree grows with enabled features. Desktop themes, desktop plugins, TUI widgets, browser caches, checkpoints, knowledge stores, and tool-specific state can also live below the Hermes home. Back up the actual home rather than relying on a hand-written list.

Keep the separation simple: secrets belong in `.env` or the platform's secret mechanism; ordinary behaviour belongs in `config.yaml`. Set restrictive permissions on the home and `.env`:

```
chmod 700 ~/.hermes
chmod 600 ~/.hermes/.env ~/.hermes/auth.json 2>/dev/null || true
```

Do not confuse `HERMES_HOME` with the operating-system `HOME`. `HERMES_HOME` selects the Hermes profile and therefore its config, memory, sessions, skills, logs, and gateway state. Host terminal subprocesses normally retain the real OS user's `HOME` so `git`, `ssh`, `gh`, npm, cloud CLIs, Claude Code, and Codex can find existing credentials. Set `terminal.home_mode: profile` only when you intentionally want a separate subprocess home, and then initialize those credentials inside the profile home.

3.1 Providers and models

Hermes separates the agent runtime from the provider that supplies model inference. Providers can be hosted APIs, Nous Portal, OpenAI-compatible endpoints, local model servers, or provider-specific integrations. The exact provider list changes; use the current model configuration guide and `hermes model` for the live inventory.

The setup wizard is safer than guessing provider-specific YAML keys. A generic environment-based example looks like this:

```
cat >> ~/.hermes/.env <<'EOF'
OPENAI_API_KEY="replace-with-a-secret"
OPENAI_BASE_URL="https://api.example.test/v1"
EOF

hermes config set model.provider openai-compatible
hermes config set model.default model-name
hermes doctor
```

The provider and model are separate settings. Set both, or use `hermes model` and the provider wizard, especially when changing an existing profile; setting only a model name can leave the previous provider selected.

For local OpenAI-compatible servers, keep the endpoint on loopback until you have deliberately designed authentication and network access. LM Studio commonly uses `http://127.0.0.1:1234/v1`; vLLM and SGLang often use an OpenAI-shaped endpoint

configured through `OPENAI_BASE_URL`. A placeholder API key may still be required by the client library even when the local server ignores it.

Model selection affects more than prose quality. Tool-call formatting, context length, reasoning behaviour, latency, cost, vision support, and provider rate limits all affect the agent loop. A model that looks good on a chat benchmark can still fail because it emits invalid tool arguments or loses the current working directory after a long tool trace.

3.2 Provider fallback and model overrides

Configure a default model for normal work, then add a deliberate fallback policy if unattended operation matters. Test fallbacks by disabling the primary provider; do not assume a configured fallback is healthy just because its name parses.

OpenAI-compatible clients often send a hard-coded model such as `gpt-4o`. Hermes intentionally treats a bare model value conservatively on `/v1/chat/completions` and `/v1/responses`: unless direct model requests are enabled, the gateway can ignore that generic name and use its configured default. A request with an explicit Hermes provider, or Hermes-native run/session APIs, follows the documented routing rules. This avoids accidental provider bypass by clients that hard-code a model field.

The API server's `GET /v1/models` is intentionally minimal. It advertises the Hermes-facing model name that a frontend needs; it is not the complete provider/model picker. Hermes-aware clients can use `/api/model/options` for richer provider-aware metadata.

3.3 Memory, skills, and durable state

Configuration, sessions, credentials, memory, skills, and logs all live under the active Hermes home. Treat that home as application state: back it up, restrict its permissions, and do not let independent writers share it casually.

3.3.1 Memory: curated continuity, not an infinite transcript

Hermes has bounded built-in memory. The default memory files are `~/.hermes/memories/MEMORY.md` for durable agent notes and `USER.md` for user preferences. They are compact and injected as a snapshot at session start, not treated as an unlimited conversation transcript. The Persistent Memory guide documents the current limits and controls.

```yaml
memory:
  memory_enabled: true
  user_profile_enabled: true
  memory_char_limit: 2200
  user_char_limit: 1375
  write_approval: true
```

With `write_approval: true`, foreground interactive writes can ask for approval and non-interactive or messaging writes can be staged for review. This is useful when memory is valuable but an unattended agent must not silently turn a transient instruction into durable behaviour.

Hermes also supports external memory provider plugins. The Memory Providers guide describes providers such as Honcho, Mem0, Hindsight, OpenViking, Holographic, RetainDB, ByteRover, and Supermemory. Only one external provider is active at a time, while built-in memory remains active alongside it. External memory creates another availability, privacy, and backup dependency; document where the data lives before enabling it.

Do not run two agent processes against one Hermes home. Both can write memory, and each can load the other's writes into its next system prompt. Use profiles for independent agents and an external memory provider when deliberate sharing is required.

3.3.2 Skills: procedural knowledge with a write boundary

Skills are on-demand knowledge documents, normally `SKILL.md` files under `~/.hermes/skills/`. Hermes reads short descriptions cheaply and loads a skill's full procedure when the task needs it. This progressive disclosure pattern lets you add detailed workflows without injecting every instruction into every request.

The Skills System guide describes bundled, installed, external, and agent-created skills. External directories can be configured when a team maintains a shared skill tree:

```
skills:
  external_dirs:
    - ~/.agents/skills
    - /home/shared/team-skills
```

External directories are not automatically read-only. If the Hermes process can write them, an agent-managed skill update can change them. For a higher-control profile, enable the skill write gate:

```
skills:
  write_approval: true
```

Then review pending changes with the documented `/skills pending`, `/skills diff`, `/skills approve`, and `/skills reject` commands. Treat skill content as executable policy: a malicious or careless skill can tell the model to exfiltrate secrets, weaken a terminal boundary, or approve a dangerous workflow.

4 Set the execution boundary

The terminal backend determines where commands execute. Hermes currently supports local, Docker, SSH, Modal, Daytona, Vercel Sandbox, and Singularity/Apptainer backends. The Terminal Backend section of Configuration is the reference for options and optional dependencies.

This table contrasts where each backend runs commands and the isolation it provides.

Backend	Execution location	Isolation posture	Good default use
`local`	The Hermes host	None beyond the OS account	Personal development with trusted prompts
`docker`	A persistent container	Namespaces, caps, resource controls	Unattended or untrusted tool loops
`ssh`	A remote host	Network and account boundary	Keep the gateway away from the work machine
`modal` / `daytona` / `vercel_sandbox`	Managed cloud workspace	Provider-managed sandbox	Ephemeral or scalable execution
`singularity`	High-performance computing (HPC) container	Cluster-compatible containment	Shared research/HPC environments

The local backend is convenient and dangerous because a successful tool call is a real host command. Set a narrow `terminal.cwd`, run Hermes as a non-root account, avoid mounting sensitive directories, and do not place broad cloud credentials in the process environment.

For a container backend, start with a deliberately limited configuration:

```yaml
terminal:
  backend: docker
  cwd: /workspace
  timeout: 180
  docker_image: python:3.12-slim
  docker_mount_cwd_to_workspace: false
  docker_run_as_host_user: true
  docker_network: false
  container_persistent: false
  container_cpu: 2
  container_memory: 2048
```

The exact resource key names can evolve; check `hermes config` and the current guide before applying a large policy file. Mounting the current working directory into `/workspace` is an

explicit access grant. Forwarding `GITHUB_TOKEN`, `NPM_TOKEN`, or cloud credentials is also an explicit access grant. Only forward variables the terminal workload actually needs.

SSH can provide a cleaner boundary than local execution:

```yaml
terminal:
  backend: ssh
  cwd: /srv/hermes-work
```

```
TERMINAL_SSH_HOST=agent-worker.example.internal
TERMINAL_SSH_USER=hermes
TERMINAL_SSH_KEY=~/.ssh/hermes_agent_key
```

The gateway host then holds messaging and provider credentials while the worker host holds the project files. This does not make the model trusted; it limits where a bad command can run.

4.1 Security baseline

Use platform allowlists and pairing rather than `GATEWAY_ALLOW_ALL_USERS=true`. Keep API and dashboard listeners on loopback unless you have an authenticated reverse proxy or private network. Give every externally reachable API server a strong bearer key. Use dashboard authentication for a non-loopback bind; the current Docker documentation explicitly fails closed for an unauthenticated public dashboard, and the old insecure bypass is not a safe deployment mechanism.

Keep `.env`, `auth.json`, and backups private. Review skills and `SOUL.md` like code. Enable memory and skill write approval for unattended or multi-user deployments. Turn on tool-loop hard stops for gateways that cannot be watched interactively:

```yaml
tool_loop_guardrails:
  hard_stop_enabled: true
  hard_stop_after:
    exact_failure: 5
    idempotent_no_progress: 5
```

The model should never be the only approval layer for sending messages, deleting files, changing credentials, or modifying a deployment.

4.2 Profiles: separate agents without separate installs

A profile is an independent Hermes home. It gives a coding agent and a research agent different configuration, credentials, personality, skills, memory, sessions, cron jobs, and gateway state while sharing the installed runtime.

```
hermes profile create coder
hermes profile create researcher --description "Reads source and external docs, writes findings."

hermes -p coder setup model
hermes -p coder config set terminal.cwd /home/me/src
hermes -p coder chat
```

```
hermes -p researcher config set terminal.cwd /home/me/research
hermes -p researcher chat
```

The profile guide explains that profile commands use `HERMES_HOME` internally. The default profile is the root `~/.hermes`; named profiles live below `~/.hermes/profiles/`. `HOME` remains the OS account home by default, which is why two profiles can intentionally share normal CLI credentials unless you opt into `terminal.home_mode: profile`.

Profile gateways need deliberate lifecycle and port planning. A deployment may run separate gateway processes, each with its own port, or use the multiplexed multi-profile gateway and route named profiles through `/p/<profile>/` on one listener. In multiplexed mode, each named profile needs its own `API_SERVER_KEY`; the default profile's key is not accepted on a named-profile route. Do not assign one port per profile unless you are actually running separate listeners. The headless `serve` backend can also serve several profiles; do not assume that it gives every client a separate `8642` listener.

5 Add one ingress surface

5.1 Messaging gateway: `hermes gateway`

The Messaging Gateway guide describes a single background gateway that can connect configured adapters such as Telegram, Discord, Slack, WhatsApp, Signal, Matrix, email, Home Assistant, Microsoft Teams, and many others. The gateway owns platform connections, per-chat sessions, typing/streaming behaviour where supported, scheduled jobs, and outbound delivery.

Configure adapters through the wizard:

```
hermes gateway setup
hermes gateway start
hermes gateway status
```

For debugging, run it in the foreground:

```
hermes gateway
```

For a user-level service on a long-lived host, use the documented install path and enable lingering if the service account must run after logout:

```
hermes gateway install
loginctl enable-linger "$USER"
hermes gateway restart
```

The exact environment variable depends on the platform. Keep tokens in `${HERMES_HOME:-~/.hermes}/.env`, and use platform-specific allowlists or pairing. These controls decide who can reach the gateway; they do not replace the profile's tool and terminal policy. Ordinary chat from an allowed user can still reach every enabled tool. `hermes send` is a useful distinction: for

many bot-token platforms it can call the platform REST endpoint directly and exit, so an inbound gateway process is not always required for an outbound one-shot message.

Do not expose the gateway's platform credentials to a second container that shares the same home. Do not run two gateways against one profile. Session files, memory, and gateway state are designed for one active writer.

5.2 OpenAI-compatible API

The API server is an HTTP adapter for applications that already speak the OpenAI format. It is not `hermes serve`, and it is not a replacement for `hermes gateway`. The current API Server documentation describes Chat Completions, Responses, run lifecycle endpoints, health checks, approvals, steering, and model discovery.

Enable it in the active profile and start the gateway process that hosts it. Using `hermes config set` avoids duplicate `.env` entries when you repeat the setup:

```
hermes config set API_SERVER_ENABLED true
hermes config set API_SERVER_KEY replace-with-a-long-random-secret
hermes gateway
```

Test the endpoint from the same host:

```
curl http://127.0.0.1:8642/v1/chat/completions \
  -H "Authorization: Bearer replace-with-a-long-random-secret" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "hermes-agent",
    "messages": [{"role": "user", "content": "Reply with one short health check."}],
    "stream": false
  }'
```

The main compatibility routes include:

```
POST /v1/chat/completions     OpenAI Chat Completions, including server-sent event (SSE) streaming
POST /v1/responses            OpenAI Responses-style stateful requests
POST /v1/runs                 Start a Hermes-native run
GET  /v1/runs/{id}             Inspect a run
GET  /v1/runs/{id}/events      Stream run events
POST /v1/runs/{id}/approval    Resolve an approval pause
POST /v1/runs/{id}/steer       Inject guidance at a tool boundary
POST /v1/runs/{id}/stop        Stop a run
GET  /v1/models                Minimal advertised model list
GET  /health                   Cheap liveness check
GET  /health/detailed          Authenticated readiness information
```

Treat caller identity and session identity as separate design decisions. A Chat Completions client normally sends the conversation history on each request; a Responses or Hermes-native run can retain server-side state. A shared frontend should map each user or workspace to an explicit session or run scope rather than letting every caller inherit one profile's unexamined history and memory. When external memory is enabled, also send a stable `X-Hermes-Session-Key` so durable memory is scoped to the intended caller or workspace.

For a tool-using client, test the approval path before enabling unattended traffic. Start with a harmless request that triggers a configured approval, confirm that the run pauses, resolve it through the documented approval endpoint, and then verify that the run resumes. A successful HTTP response proves only that the listener and key work; it does not prove that tool policy or approvals are effective.

The HTTP API can invoke terminal, file, web, memory, and skill tools. That makes an API key equivalent to permission to drive an agent, subject to the profile's tool and terminal policy. Keep the default bind at `127.0.0.1`, tunnel it over SSH when practical, and use a narrow cross-origin resource sharing (CORS) allowlist only for browser clients. If you bind `0.0.0.0`, require `API_SERVER_KEY`, put the service behind a private network or authenticated reverse proxy, and do not rely on CORS as authentication.

Port conflicts are especially common with profiles and containers. For separate gateway processes, give each API-enabled profile its own port. For a multiplexed gateway, keep one listener and use the documented profile paths and per-profile keys. Keep per-profile environment variables in the profile's own `.env` rather than setting one container-wide value for every profile.

5.3 Desktop and `hermes serve`

`hermes serve` launches the headless backend used by Hermes Desktop and remote desktop-style clients. It serves the `tui_gateway` JSON-RPC/WebSocket protocol and related HTTP endpoints. Desktop launches this process locally by default or connects to a remote one when configured. It does not open the browser dashboard and does not configure messaging adapters.

Run it directly when you need a backend for a remote Desktop client or want to debug the backend independently of Electron:

```
hermes serve --host 127.0.0.1 --port 9119
```

On current installations the common listener is `127.0.0.1:9119`, but the process startup output is authoritative. A Desktop-managed backend may choose another available port and announces readiness in a line such as `HERMES_BACKEND_READY port=...`. Use `hermes serve --help` for the runtime flags supported by the installed version.

The web dashboard is a related but different client surface. `hermes dashboard` starts the browser dashboard backend and UI; `hermes serve` shares the server foundation but selects the headless path. A dashboard or `serve` backend does not start `hermes gateway`, and the messaging gateway does not make a Desktop WebSocket client available.

Treat a remote `serve` endpoint as an administrative agent backend. The safest remote pattern keeps it on loopback and uses an SSH tunnel or private VPN path. If you bind it to a private interface, configure the supported dashboard authentication provider and signing secret before starting it; a non-loopback bind without an authentication gate should fail closed. Do not publish `9119` directly to the public internet. If a reverse proxy terminates TLS, preserve WebSocket upgrades and forward the authentication mechanism expected by the installed Hermes version. Verify the exact readiness line and complete one authenticated Desktop connection before adding a gateway or API client.

6 Deploy and supervise

There are two different container questions:

1. **Run Hermes inside a container.** The official image stores the agent's mutable data under `/opt/data`, normally backed by a host `~/.hermes` mount.
2. **Use a container as the terminal backend.** Hermes itself runs on the host, while tool commands execute in a persistent sandbox container.

Confusing these leads to missing state or an accidentally unsandboxed terminal. The official Docker guide covers both patterns.

6.1 Run Hermes in a container

The official Docker image uses `nousresearch/hermes-agent`. Configure the mounted home first:

```
mkdir -p ~/.hermes
docker run --rm -it \
  -v ~/.hermes:/opt/data \
  nousresearch/hermes-agent setup
```

Run a supervised gateway with the API server enabled and bound to the container interface. The host-side publication remains loopback-only:

```
docker run -d \
  --name hermes \
  --restart unless-stopped \
  -v ~/.hermes:/opt/data \
  -p 127.0.0.1:8642:8642 \
  -e API_SERVER_ENABLED=true \
  -e API_SERVER_HOST=0.0.0.0 \
  -e API_SERVER_KEY="replace-with-a-long-random-secret" \
  nousresearch/hermes-agent gateway run
```

The dashboard is a separate, authenticated service. Do not add `HERMES_DASHBOARD=1` to a public or remotely reachable container until you have configured one of the supported dashboard authentication providers and its signing secret; current Hermes fails closed for an

unauthenticated non-loopback bind. Follow the Docker dashboard instructions and verify the login gate before exposing `9119`.

The image is intended to be upgraded independently of `/opt/data`. Keep the image tag pinned for production rather than accepting an unreviewed `latest` update. The official image now supervises gateway services with s6; inspect `docker logs` and the mounted profile logs when debugging, and remember that two containers must never write the same Hermes home concurrently.

6.1.1 Podman

Rootless Podman is a reasonable Docker-compatible runtime for the official image, but validate the exact image, networking, user namespace, and volume behaviour in your environment. Use `podman` in place of `docker`, publish loopback addresses explicitly, and add `:Z` on SELinux systems when the host policy requires relabeling:

```
podman run -d \
  --name hermes \
  --replace \
  -v "$HOME/.hermes:/opt/data:Z" \
  -p 127.0.0.1:8642:8642 \
  -e API_SERVER_ENABLED=true \
  -e API_SERVER_HOST=0.0.0.0 \
  -e API_SERVER_KEY="replace-with-a-long-random-secret" \
  nousresearch/hermes-agent gateway run
```

Rootless UID mapping can make a mounted directory appear owned by an unexpected host UID. Check `podman unshare`, the directory permissions, and the runtime user before blaming Hermes. If Hermes uses a container terminal backend, the configuration can select the runtime binary with the documented `HERMES_DOCKER_BINARY` override; test whether the installed Hermes version accepts Podman for every Docker-specific option you enable.

Do not assume a rootless container has access to the host Docker socket, GPU, audio devices, or browser sandbox. Grant those resources explicitly, and document the resulting security tradeoff. For a remote Desktop backend, publish or proxy the `serve` port separately from the API port; publishing `8642` does not make the JSON-RPC/WebSocket backend available.

6.1.2 Compose shape

The minimal Compose shape below exposes only the authenticated API. Add the dashboard only after configuring its authentication gate from the official guide.

```
services:
  hermes:
    image: nousresearch/hermes-agent:REPLACE_WITH_TESTED_TAG
    container_name: hermes
    restart: unless-stopped
    command: gateway run
```

```
    ports:
      - "127.0.0.1:8642:8642"
    volumes:
      - "${HOME}/.hermes:/opt/data"
    environment:
      API_SERVER_ENABLED: "true"
      API_SERVER_HOST: "0.0.0.0"
      API_SERVER_KEY: "${API_SERVER_KEY:?set API_SERVER_KEY in the environment}"
```

Do not copy `API_SERVER_KEY` into a committed Compose file; provide it from an untracked environment file or secret manager. Test the exact image tag before rollout, and never run two containers against the same Hermes home concurrently.

7 Operate and diagnose

Use the smallest diagnostic surface that answers the question:

```
hermes --version
hermes doctor
hermes config
hermes profile list
hermes gateway status
hermes logs --follow
```

For API clients, test liveness before testing a model:

```
curl -fsS http://127.0.0.1:8642/health
curl -fsS http://127.0.0.1:8642/v1/models \
  -H "Authorization: Bearer $API_SERVER_KEY"
```

For Desktop, verify the `hermes serve` process and the exact readiness port before inspecting model credentials. Check the desktop log and the backend log for authentication failures, WebSocket upgrade failures, stale process IDs, or a port already in use.

For a messaging incident, check in this order: platform token and adapter enablement, gateway process status, pairing/allowlist, session routing, provider health, then tool execution. For an API incident, check in this order: listener and bind address, bearer key, request path, concurrency cap, provider/model, then tool policy. For a tool incident, identify the terminal backend first; the same agent can be safe in Docker and dangerous on `local`.

Keep logs on persistent storage but treat them as sensitive. They can contain prompts, paths, provider errors, and operational metadata even when secrets are redacted. Rotate or retain them according to the privacy needs of your deployment.

Updates change both the runtime and bundled skills. Review the CLI reference, take a backup before a major update, then run `hermes doctor` and a small tool-free prompt before re-enabling unattended gateways.

8 Backup, restore, and recover

Hermes has two useful backup scopes. `hermes backup` captures the whole Hermes home, including profiles and credentials, while `hermes profile export` creates a portable single-profile archive with `.env` and `auth.json` excluded. The official profile command reference and CLI reference describe the current behaviour.

8.1 Full machine backup

The full backup is appropriate for disaster recovery or moving the installation. It contains secrets, so protect the archive like a password vault:

```
umask 077
hermes backup --output "$HOME/hermes-backup-$(date +%Y%m%d-%H%M%S).zip"
chmod 600 "$HOME"/hermes-backup-*.zip
```

The backup command uses SQLite's backup API for a consistent database snapshot. It excludes the Hermes source checkout and transient SQLite sidecars. For a planned migration, stop every writer first: gateway, dashboard, `hermes serve`, cron workers, and any container or service using the same home. Confirm that no Hermes process still has the home open before the cutover.

```
hermes gateway stop
# Also stop dashboard/serve, cron, and any container or service using ~/.hermes.
hermes backup --output /secure/location/hermes-full.zip
```

Check the backup command's exit status and output. A partial archive can be left on disk when files could not be added; do not treat the presence of a `.zip` file as proof of a complete backup. Inspect the archive, record any skipped files, and keep the previous known-good backup until a restore test succeeds.

Restore only after installing a compatible Hermes runtime and preserving a copy of the destination home:

```
mv ~/.hermes ~/.hermes.before-restore
mkdir -p ~/.hermes
hermes import /secure/location/hermes-full.zip
hermes doctor
hermes profile list
```

The exact import confirmation and path behaviour can change; read `hermes import --help` before an overwrite. Re-authenticate providers or messaging platforms if tokens were intentionally excluded or invalidated. Start one profile at a time and verify that the restored API bind and gateway adapters are not colliding with an old process. Run `hermes doctor`, a tool-free prompt, an authenticated API health check, and a controlled channel smoke test before reconnecting unattended traffic. Keep `~/.hermes.before-restore` until those checks pass.

8.1.1 Profile handoff

For sharing a profile with another machine or person, prefer the credential-stripping export:

```
hermes profile export researcher \
  --output ./researcher-2026-08-22.tar.gz

hermes profile import ./researcher-2026-08-22.tar.gz \
  --name researcher-restored
```

Inspect archives before sharing them. Memory, sessions, user preferences, skills, and `SOUL.md` can contain private material even when API keys are removed. A profile distribution in a Git repository is a different mechanism: it versions a reviewed agent bundle while keeping each install's credentials and user data local.

8.1.2 Recovery checklist

When Hermes fails after an update, preserve logs and the backup before deleting state. Check the installed version, run `hermes doctor`, validate YAML, confirm that the launcher points at the managed virtual environment, and test the provider with a minimal prompt. If the problem is profile-specific, create a temporary profile to separate runtime failure from corrupted state. Restore the original profile only after confirming which artifact is damaged.

9 Reference and alternatives

OpenClaw and Hermes appear in the same local-first assistant discussions because they can both sit between people, models, tools, and messaging channels. They are not identical products, and "Hermes" in this document is not a model family.

At a high level, OpenClaw is an assistant orchestration/control-plane platform organized around its gateway, channels, sessions, routing, tool policy, and provider integrations. Hermes Agent is an agent runtime from Nous Research that combines interactive clients, tools, skills, memory, profiles, messaging adapters, an OpenAI-compatible API server, and a Desktop/headless backend. Both can be deployed as a long-running personal or team assistant, but their command names, configuration layouts, protocols, and operational assumptions differ.

Dimension	Hermes Agent	OpenClaw
Primary identity	Tool-using agent runtime with CLI/TUI, profiles, skills, memory, messaging, API, and Desktop backend	Assistant orchestration platform and gateway

Dimension	Hermes Agent	OpenClaw
Messaging process	`hermes gateway`	OpenClaw gateway and channel adapters
OpenAI-shaped integration	Optional API server, normally `8642`	Provider/API integrations according to OpenClaw configuration
Desktop/headless backend	`hermes serve`, JSON-RPC/WebSocket, commonly `9119`	OpenClaw's own control/UI surfaces and APIs
Durable state	`~/.hermes`, `HERMES_HOME`, profile directories, SQLite/session files, skills and memory	OpenClaw state/workspace/config layout
Terminal isolation	Local, Docker, SSH, Modal, Daytona, Vercel Sandbox, Singularity	OpenClaw execution and sandbox policy
Model role	Hermes routes to configured providers; it is not the Nous Hermes model	OpenClaw also routes to configured providers; it is not a model server
Main operational question	Which Hermes surface and profile is running, and where do tools execute?	Which gateway, channel, provider, session, and sandbox policy is active?

9.1 When Hermes is the better fit

Choose Hermes when you want the Nous Research agent runtime, its skill and memory model, profile isolation, terminal-backend choices, a native Desktop client, or a direct OpenAI-compatible agent endpoint. It is particularly attractive when the distinction between a model response and a tool-using, stateful agent is central to your application.

9.2 When OpenClaw is the better fit

Choose OpenClaw when its gateway/channel orchestration, ecosystem integrations, routing model, sandbox controls, or existing deployment conventions match your needs. If your team already operates OpenClaw, replacing it with Hermes solely because both can receive chat messages is not a technical comparison; compare the control-plane features and recovery model you actually use.

9.3 When they can be combined

They can occupy different layers: OpenClaw can be the outward channel/control plane while Hermes or another backend supplies agent behaviour, or Hermes can expose an API that a separate application consumes. A combined deployment adds network, authentication, session, and duplicate-tool risks. Define which system owns identity, model selection, memory, terminal execution, and outbound messaging before connecting them. Do not let both systems independently answer the same channel or write the same state directory.

9.4 Operational reference

9.4.1 Surface-selection checklist

```
Need Telegram/Discord/Slack inbound messages?  Start and configure hermes gateway.
Need Open WebUI or curl over HTTP?             Enable the API server on 8642.
Need Hermes Desktop or remote JSON-RPC?        Run/connect to hermes serve, often 9119.
Need browser dashboard administration?         Use hermes dashboard with authentication.
Need command isolation?                         Configure terminal.backend, separately.
Need independent identity and memory?          Create a profile, not another process on one home.
```

9.4.2 Hardening checklist

1. Keep `8642` and `9119` on loopback or a private network by default.
2. Require a strong API bearer key and use a narrow CORS allowlist for browser clients.
3. Configure dashboard authentication before any non-loopback bind.
4. Use gateway allowlists and pairing; do not enable all users casually.
5. Run as a non-root OS account.
6. Prefer Docker or SSH terminal execution for unattended agents.
7. Set a narrow `terminal.cwd` and avoid broad host mounts.
8. Keep secrets in `.env` or a secret manager, never in Git or Compose files.
9. Review agent-written skills and memory, enabling write approval when needed.
10. Back up before updates and verify restored profiles on a test path.

9.4.3 Troubleshooting map

Use this symptom map to choose the first diagnostic area.

Symptom	First place to look
Telegram/Discord does nothing	`hermes gateway status`, adapter token, pairing, allowlist
Open WebUI says connection refused	`API_SERVER_ENABLED`, bind/port, gateway process, container publish

Symptom	First place to look
API returns unauthorized	`API_SERVER_KEY`, profile `.env`, proxy headers, wrong port/profile
Desktop cannot connect	`hermes serve`, readiness port, WS upgrade, backend auth, VPN/reverse proxy
Tools run on the wrong machine	`terminal.backend`, `terminal.cwd`, `HERMES_HOME` vs `HOME`
Agent remembers too much or too little	memory limits, profile selection, external provider status
Skill change appears unexpectedly	bundled sync, external directory writability, skill write gate
Two profiles collide	duplicate API port, shared home, duplicate gateway process
Container loses everything after restart	missing `/opt/data` mount or wrong host path/permissions

9.5 Reference links

- Hermes Agent documentation
- NousResearch/hermes-agent repository
- Installation
- Quickstart
- Configuration
- Providers and models
- Messaging Gateway
- OpenAI-compatible API Server
- Desktop and remote backends
- Docker
- Security
- Persistent Memory
- Memory Providers
- Skills System
- Profiles
- Profile command reference
- CLI command reference
- Gateway internals

- OpenClaw repository

9.6 Closing perspective

Hermes becomes easier to operate once you stop treating "the server" as one thing. The messaging gateway is for platform connections. The API server on `8642` is for OpenAI-shaped HTTP clients. `hermes serve`, commonly on `9119`, is for Desktop and headless JSON-RPC/WebSocket clients. Profiles and `~/.hermes` provide continuity, while terminal backends define where the agent's actions land.

The safest deployment is therefore explicit: one profile per independent agent, one clear owner for each channel, one documented terminal boundary, loopback-first network exposure, authenticated remote access, and a tested backup. The model can be capable, but the surrounding system must still make unsafe actions difficult and recoverable.